

The strategies in this part will help you find an entry point quickly. Try a shortcut, search the code, search the internet, ask someone else, attach a debugger, or look at a crash dump.

4. Try Shortcuts

How? Based on your experience, you can make educated guesses about which areas of your program's source relate to your bug's symptoms. You can also check systems that have repeatedly had bugs and confirm they're not at fault, initially ignoring the ones with responsibilities that probably don't relate to your particular problem.

Why? Once you know you need to jump to a specific point in your program's sources, getting there doesn't take much time: With most integrated development environments (IDEs) or editors, it takes only a moment to jump from line 20 to line 10,438 in a file. One method of identifying the part you need is narrowing down the target area by ruling out larger sections. If you can disregard most of the program's sources, you'll have a better idea of where to look for your target line. The presented shortcuts can work either way: discovering that a system is related to your bug or ruling it out.

Shortcuts are alluring. It's difficult to resist skipping all the hard work and getting a task done a lot quicker than your team would typically expect. However, you must be careful not to fall into the trap of repeatedly trying to find shortcuts without making any progress. Likewise, it's easy to get too focused on one where the line of inquiry doesn't lead to the spot that's at fault. If shortcuts don't quickly get you where you need to go, they slow you down because they keep you from beginning detailed investigations. In the worst case, you're mistaken and spend time looking into a supposed lead that turns out to be a dead end.

Throughout your work, keep checking to make sure you're on the right track. Does the spot you found via a shortcut really fit the bug you're working on? Is it reasonable to conclude that this code area is related to the observed symptoms? Or might this code be unrelated, meaning you can rule out this area as the source of the problem? Make sure to stay focused.

IN PRACTICE: AVOID UNLIKELY CULPRITS

On one project, my colleague was working on a bug related to network communication: Something was going wrong with our HTTP requests. The project used curl, a library that implements several network protocols. My colleague followed the bug's traces and spent a lot of time trying to debug curl itself because that seemed like the logical source of the problem.

exception isn't handled elsewhere. Such a mechanism is often used to let assertions trigger crash dumps (see "Instrument the Program to Report Problems" on page 62).

Get Information from a Crash Dump

Acquiring a crash dump for a bug you're working on is almost as good as catching it in the debugger yourself. You can load the file into your debugger and "run" it there to get most of the same information (most importantly the call stack). Some crash dumps may contain variable values and program memory, which is hugely valuable but increases file size. If yours doesn't, you have to go by the logic's state and guess which variable may have had which value.

The biggest drawback of crash dumps is that they're not interactive. You can't see how the logic evolves over time. If you were actually debugging, you would potentially have stepped through what came before, and you might be able to continue or manipulate the instruction pointer to hop over the problematic code. However, if you had only caught the hardware exception when it had already been thrown, you wouldn't be able to continue past it anyway.

What you see in the debugger may also be challenging to analyze because the displayed code isn't necessarily what's being executed. Crash dumps often stem from end-user builds created with optimizations enabled, which means that the compiler transformed the code quite a bit. If you repeatedly observe crashes in a specific function but can't figure out what's going on, it may be worthwhile to disable optimizations for that specific function and ship a build with this change (see "Step Through the Code" on page 100), being careful about resulting performance issues.

Implement Crash Dumps

First, you must decide the right time to introduce crash dump support to your program. Since the goal of doing so is typically to catch users' issues, it may happen only shortly before release.

However, having crash dumps in place early during development is also hugely beneficial. It makes it easier for you to weed out crashes before the release and allows your testers to provide more hard data when creating bug reports (see "What Makes a Great Bug Report?" on page 11). Having too many crashes during development also slows down the whole team.

Adding support isn't much work in the sense of writing a lot of code because it doesn't require you to create complicated new systems, define data structures, or implement complicated algorithms. The complexity comes from dealing with highly platform-specific features that vary between operating systems and hardware architectures.

Additionally, you're operating outside the realm of functionality a program typically tackles. You have to write code that can run after your program has caused a catastrophic error and work flawlessly when there's not much in the environment it can rely on.

Implement Crash Dumps for Your Platform

Implementing crash dumps is highly platform-dependent, with significant differences between various operating systems on PCs, mobile devices, and proprietary devices. This section shows an example using C on Windows.

The first step is to react to the crash. Very early on in your main function, you have to set up a handler for unhandled exceptions:

```
int main(int argc, char** argv) {
    SetUnhandledExceptionFilter(handle_exception);
    --snip--
```

The `handle_exception()` function implements the creation of the crash dump, which also conveniently stores the data in a file:

```
#include <windows.h>
#include <Dbghelp.h>
#include <tchar.h>

typedef BOOL (WINAPI *MINIDUMPWRITEDUMP)(HANDLE hProcess,
    DWORD dwPid, HANDLE hFile, MINIDUMP_TYPE DumpType,
    CONST PMINIDUMP_EXCEPTION_INFORMATION ExceptionParam,
    CONST PMINIDUMP_USER_STREAM_INFORMATION UserStreamParam,
    CONST PMINIDUMP_CALLBACK_INFORMATION CallbackParam);

LONG WINAPI handle_exception(struct _EXCEPTION_POINTERS* exception_info) {
    HMODULE lib = LoadLibrary(_T("dbghelp.dll"));
    MINIDUMPWRITEDUMP pDump =
        (MINIDUMPWRITEDUMP)GetProcAddress(lib, "MiniDumpWriteDump");

    HANDLE file = CreateFile(_T("mydump.dmp"), GENERIC_WRITE,
        FILE_SHARE_WRITE, NULL, CREATE_ALWAYS,
        FILE_ATTRIBUTE_NORMAL, NULL);

    _MINIDUMP_EXCEPTION_INFORMATION ExInfo;
    ExInfo.ThreadId = GetCurrentThreadId();
    ExInfo.ExceptionPointers = exception_info;
    ExInfo.ClientPointers = FALSE;

    pDump(GetCurrentProcess(), GetCurrentProcessId(),
        file, MiniDumpNormal, &ExInfo, NULL, NULL);
    CloseHandle(file);

    return EXCEPTION_CONTINUE_SEARCH;
}
```

Now that you have the crash dump in a file, you'll be able to load it up in a debugger and look at what has happened. As it is, the process involves asking the users or testers to share the file with you for every incident.

An ideal solution would share it automatically. However, the more logic that must run in the event of a crash, the more can go wrong. Imagine your program operates on a device that runs out of memory, and the memory

allocation function returns a null pointer. Because you don't check the result but just write to the provided address, your program crashes. In this common scenario, you'll very quickly encounter an access violation.

Having to run extensive logic in a catastrophic event isn't a great idea. That's why there's a more reliable but also more complex solution: You can implement your crash dump handler as an out-of-process handler. Instead of doing any extensive computing, the crashed program delegates the crash handling to another process. This may spawn when a crash takes place or run continuously as a kind of watchdog. The latter option simplifies response when a program does fail, because you can fully prepare the handler and have all the memory it requires reserved in advance. Then it just needs to create the crash dump and perform any further processing. (It would also be nice to implement a little UI where the user could describe what they were doing when the crash occurred.)

Generate and Store Symbol Files

The significant advantage of crash dumps is that they work with end-user builds, but those are typically *stripped builds*, meaning the executable no longer contains any debugging information. You won't see any variable names, function names, source filenames, or line numbers, but only assembly instructions and memory addresses. That's still better than nothing, but it can be very hard or impossible to figure out what part of your source code is to blame when you have such limited information.

While your executable should get stripped during the process of preparing a distribution build, you should also make sure to generate a symbol file that solely contains the otherwise missing debugging information. You may even have multiple symbol files for a single build, typically one for each executable and one for each shared library (*.dll* or *.so* files). You must archive these so you have them readily available if you need to debug a crash dump.

One symbol file is only ever associated with one particular build, so make sure you know which belongs to which. Assuming that the builds you distribute have a version number, a working simple approach is to label a directory accordingly and store the files there. Symbol files contain whatever information is needed to correlate the code and data in the executable with the source code. For example, the *.pdb* format used by Windows contains the following:

Symbolic information Function or variable names and their memory locations

Source code information Filenames and paths, line number information (mapping memory addresses to specific lines of code)

Type information Structure layouts, function signatures, type definitions (enums, typedefs, and user-defined types)

Build information Compiler settings, version information, and timestamps related to the build process

When debugging a crash dump, you may see that parts of the call stack are inside a third-party library. When you don't build libraries yourself, you need symbol files to debug them effectively, so it's standard procedure to ship them together. Even Microsoft does it. Symbol files can get quite large. Because they contain a lot of text, they compress well, but you have to take care of the compression yourself. Even the compressed version can still take up several gigabytes, which you'll have to store for each release until you're sure you won't have to deal with its crash dumps anymore.

IN PRACTICE: WATCH OUT FOR EXCESSIVE FILE SIZES

One project I worked on used a third-party framework that consisted of multiple `.dll` files. It came with multiple plug-ins, each built as one `.dll`. The symbol files for all of them exceeded 60GB in total, and storing this for each nightly build was just way too much. We decided to store only the symbols for our executable and exclude those of any third-party libraries.

Choose the Right Type of Crash Dump

Most crash dumps contain exception information (the reason for the crash), a call stack that includes registers, a list of threads, a list of modules, metadata about the CPU and operating system, and any active operating system handles (references to files, synchronization primitives, processes, and so on). They also include memory, which contributes the most to their size. You can leave out all or some of it, but you may inadvertently exclude important information by doing so.

On Windows, you can choose between a *full dump*, which includes all the accessible memory, or a *mini dump*, which only contains memory that local variables on the stack directly or indirectly reference. You can specify what data you want when calling `MiniDumpWriteDump()`.

So far, all of the discussion has been about CPU-side issues, but graphics processing unit (GPU) crash dumps also exist on some proprietary systems. Nvidia's GPUs, for instance, offer Nsight Aftermath (<https://developer.nvidia.com/nsight-aftermath>).

Turn Crash Dumps into Actionable Data

Once you implement crash dump creation, you should look into access. Make it as easy as possible to get the most information from your users. At the very least, the program should tell them that a crash occurred and the location of the crash dump. It could do this the next time a user starts it after a failure.

Your software should also share those crash dumps automatically with you. Shared network directory storage is straightforward and lets you regularly pick up the files.

A better solution is to upload them to a shared online service. This solution works for anyone independent of where they are in the world and which network they're on.

Once you get an online service that collects your crash dumps, you have many more possible options than just storing them. The service can analyze your files and extract information so it can display them conveniently and you can avoid opening each one just to see where the program failed. Note that the online service also needs access to associated symbols for each executable.

The next step is to group crashes that happened in the same or a similar place. That gives you more data for each bug and lets you draw statistical patterns, such as which crashes occur most often, how stable each release is, and whether a problem recurs after being marked fixed.

These kinds of statistics will improve your program's overall quality by helping you focus on important issues. Looking at the call stack and crash frequency in a dashboard makes your debugging process easier.

To facilitate this workflow even more, you may want to add additional information such as the program's log (see "Instrument the Program to Report Problems" on page 62) or let the user specify what they were doing when the crash occurred. You should track information like the CPU type, graphics card, and amount of installed RAM. Status reports like the amount of free memory at the time of the crash can also be helpful. Patterns from all this data could reveal information about the bug's cause, indicating factors such as out-of-memory problems or specific CPU issues.

Use Third-Party Tools to Simplify Crash Reporting

While the basic implementation of crash dumps is relatively straightforward, it can quickly become complex if you want one that works across all your supported platforms, collates reports, and provides actionable insights. Since you don't often use crash reporting as a selling point, it's rare to spend much time on it.

Luckily, third-party libraries and services can take most of the complexity off your hands. Such solutions can have a wide variety of feature sets. Some only generate crash dumps, while others offer full-fledged online features that provide lots of statistics and insights.

The very first step you should take is checking whether your target platforms already offer such services. Quite a few proprietary platforms do. They'll have the most knowledge about their low-level components and access to data and features that might not be available to you.

Windows Error Reporting (WER) can generate a crash dump for any program (<https://learn.microsoft.com/en-us/windows/win32/wer/windows-error-reporting>) and automatically upload it if you've signed your software properly. Getting access to these reports can be a bit tricky, though.

Open source libraries can also take most of the work off your hands. Google has two: the bare-bones, in-process Breakpad (<https://chromium.googlesource.com/breakpad/breakpad>) and Crashpad (<https://chromium.googlesource.com/crashpad/crashpad>), which generates crash dumps outside processes and facilitates uploading and sharing.

More full-featured commercial services are available as well, such as Backtrace (<https://backtrace.io>), Firebase Crashlytics (<https://firebase.google.com/products/crashlytics>), and Sentry (<https://sentry.io/>).

Summary

By now you've tried a range of ways to find where your bug lives: educated guesses based on experience, internet and codebase searches, a trip through your version control history, the debugger, and a conversation or two with people who know the code. Any one of them might have handed you an entry point, the specific spot in the source worth digging into. With that spot in mind, you're ready to stop searching and start examining. Part III is where you look closely.